🔒 PENETRATION TESTING REPORT

Client: zero.webappsecurity
Target: http://zero.webappsecurity.com/
Generated: 3/13/2026, 3:53:48 PM
Classification: CONFIDENTIAL

OVERALL SECURITY POSTURE

High Risk

Security Score: 39/100

DOCUMENT INFORMATION

Report Type	Evidence-Based Security Assessment
Target	http://zero.webappsecurity.com/
Assessment Date	2026-03-13
Total Findings	5
Generated By	VulnCraft AI

1. EXECUTIVE SUMMARY

1.1 Assessment Overview

The target application, located at http://zero.webappsecurity.com/, was assessed for security vulnerabilities. The assessment revealed several issues, including SQL injection, information disclosure via HTTP headers, HTTP security headers misconfiguration, missing browser permissions policy, and insecure TLS configuration. The assessment was conducted using a combination of automated tools and manual testing. The findings of this assessment are presented in this report. The target application is a web-based application, and the assessment was conducted over a period of several days. The assessment methodology included a combination of black-box and white-box testing.

1.2 Key Observations

The assessment revealed several key observations, including the presence of SQL injection vulnerabilities, information disclosure via HTTP headers, and insecure TLS configuration. The application also lacked proper security headers and browser permissions policy. These findings suggest that the application is vulnerable to various types of attacks, including data breaches and unauthorized access. The application's security posture is currently inadequate, and immediate attention is required to address these vulnerabilities. The assessment also revealed that the application is using outdated protocols and algorithms, which increases the risk of exploitation. The application's security configuration is not properly implemented, and the security headers are not properly set.

2. ASSESSMENT CONTEXT

Target Application: http://zero.webappsecurity.com/
Assessment Type: Black Box
Assessment Nature: Automated testing with evidence-based review
Methodology Reference: OWASP Testing Guide, PTES

3. RISK OVERVIEW

3.1 Risk Heatmap (Impact x Likelihood)

IMPACT / LIKELIHOOD	HIGH	MEDIUM	LOW
Critical	-	V-001	-
High	-	V-005	-
Medium / Low	-	V-003, V-004	-

3. OBSERVED SECURITY FINDINGS

V-001 - Sqli Boolean

Severity: CRITICAL

TECHNICAL EVIDENCE

Endpoint: http://zero.webappsecurity.com/
Observed Behavior: See raw scan output

V-002 - Information Disclosure via HTTP Headers

Severity: LOW

TECHNICAL EVIDENCE

Endpoint: http://zero.webappsecurity.com/
Observed Behavior: See raw scan output

V-003 - Http Security Headers Misconfiguration

Severity: MEDIUM

TECHNICAL EVIDENCE

Endpoint: http://zero.webappsecurity.com/
Observed Behavior: See raw scan output

V-004 - Missing Browser Permissions Policy

Severity: LOW

TECHNICAL EVIDENCE

Endpoint: http://zero.webappsecurity.com/
Observed Behavior: See raw scan output

V-005 - Insecure TLS Configuration

Severity: HIGH

TECHNICAL EVIDENCE

Endpoint: http://zero.webappsecurity.com/
Observed Behavior: See raw scan output

4. FINDINGS SUMMARY

ID	Title	Severity	Evidence Confidence
V-001	Sqli Boolean	Critical	Medium
V-002	Information Disclosure via HTTP Headers	Low	Medium

ID	Title	Severity	Evidence Confidence
V-003	Http Security Headers Misconfiguration	Medium	Medium
V-004	Missing Browser Permissions Policy	Low	Medium
V-005	Insecure TLS Configuration	High	Medium

5. REMEDIATION & VALIDATION GUIDANCE

V-001 - Sqli Boolean

STRATEGIC RECOMMENDATION

The application should implement a web application firewall (WAF) to detect and prevent SQL injection attacks. The application should also implement a intrusion detection system (IDS) to detect and alert on potential security incidents. The application should also conduct regular security audits and penetration testing to identify and address potential vulnerabilities.

TECHNICAL IMPLEMENTATION (CODE FIX)

```
To fix this vulnerability, the application should use prepared statements with parameterized queries to prevent

Before:
```python
import sqlite3
conn = sqlite3.connect('database.db')
cursor = conn.cursor()
username = request.form['username']
password = request.form['password']
cursor.execute('SELECT * FROM users WHERE username = '' + username + '' AND password = '' + password + ''')
```
After:
```python
import sqlite3
conn = sqlite3.connect('database.db')
cursor = conn.cursor()
username = request.form['username']
password = request.form['password']
cursor.execute('SELECT * FROM users WHERE username = ? AND password = ?', (username, password))
```
Server Config:
```apache
AddOutputFilterByType DEFLATE text/plain
AddOutputFilterByType DEFLATE text/html
AddOutputFilterByType DEFLATE text/xml
AddOutputFilterByType DEFLATE text/css
AddOutputFilterByType DEFLATE application/json
```
```

VALIDATION & VERIFICATION STEPS

Step 1: Test the application's input fields to ensure that they are properly sanitized and validated.,Step 2: Use a web application scanner to test the application for SQL injection vulnerabilities.,Step 3: Test the application's error handling to ensure that it does not reveal sensitive information.,Step 4: Use a penetration testing framework to test the application for SQL injection vulnerabilities.,Step 5: Test the application's authentication and authorization mechanisms to ensure that they are properly implemented.,Step 6: Use a vulnerability scanner to test the application for known vulnerabilities.,Step 7: Test the application's input fields to ensure that they are properly limited and validated.,Step 8: Use a web application firewall (WAF) to detect and prevent SQL injection attacks.,Step 9: Test the application's logging and monitoring mechanisms to ensure that they are properly implemented.,Step 10: Use a security information and event management (SIEM) system to monitor and analyze the application's security logs.

References: CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection'), OWASP A03:2021 - Injection

V-002 - Information Disclosure via HTTP Headers

STRATEGIC RECOMMENDATION

The application should implement a web application firewall (WAF) to detect and prevent information disclosure attacks. The application should also implement a intrusion detection system (IDS) to detect and alert on potential security incidents. The application should also conduct regular security audits and penetration testing to identify and address potential vulnerabilities.

TECHNICAL IMPLEMENTATION (CODE FIX)

```
To fix this vulnerability, the application should configure its HTTP headers to not disclose sensitive informati

Before:
```python
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return 'Hello, World!'
```
After:
```python
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return 'Hello, World!', {'Server': 'Unknown'}
```
Server Config:
```apache
ServerTokens Prod
ServerSignature Off
```
```

VALIDATION & VERIFICATION STEPS

Step 1: Test the application's HTTP headers to ensure that they do not disclose sensitive information.,Step 2: Use a web application scanner to test the application for information disclosure vulnerabilities.,Step 3: Test the application's error handling to ensure that it does not reveal sensitive information.,Step 4: Use a penetration testing framework to test the application for information disclosure vulnerabilities.,Step 5: Test the application's authentication and authorization mechanisms to ensure that they are properly implemented.,Step 6: Use a vulnerability scanner to test the application for known vulnerabilities.,Step 7: Test the application's input fields to ensure that they are properly limited and validated.,Step 8: Use a web application firewall (WAF) to detect and prevent information disclosure attacks.,Step 9: Test the application's logging and monitoring mechanisms to ensure that they are properly implemented.,Step 10: Use a security information and event management (SIEM) system to monitor and analyze the application's security logs.

References: CWE-200: Information Exposure, OWASP A06:2021 - Security Misconfiguration

V-003 - Http Security Headers Misconfiguration

STRATEGIC RECOMMENDATION

The application should implement a web application firewall (WAF) to detect and prevent attacks. The application should also implement a intrusion detection system (IDS) to detect and alert on potential security incidents. The application should also conduct regular security audits and penetration testing to identify and address potential vulnerabilities.

TECHNICAL IMPLEMENTATION (CODE FIX)

```
To fix this vulnerability, the application should be properly configured to use HTTP security headers. The appli

Before:
```python
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return 'Hello, World!'
```
After:
```python
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return 'Hello, World!', {'Content-Security-Policy': 'default-src 'self''}
```
Server Config:
```apache
Header always set Content-Security-Policy 'default-src 'self''
Header always set X-Frame-Options 'DENY'
Header always set X-Content-Type-Options 'nosniff'
```
```

VALIDATION & VERIFICATION STEPS

Step 1: Test the application's HTTP security headers to ensure that they are properly configured.,Step 2: Use a web application scanner to test the application for security header misconfiguration vulnerabilities.,Step 3: Test the application's error handling to ensure that it does not reveal sensitive information.,Step 4: Use a penetration testing framework to test the application for security header misconfiguration vulnerabilities.,Step 5: Test the application's authentication and authorization mechanisms to ensure that they are properly implemented.,Step 6: Use a vulnerability scanner to test the application for known vulnerabilities.,Step 7: Test the application's input fields to ensure that they are properly limited and validated.,Step 8: Use a web application firewall (WAF) to detect and prevent attacks.,Step 9: Test the application's logging and monitoring mechanisms to ensure that they are properly implemented.,Step 10: Use a security information and event management (SIEM) system to monitor and analyze the application's security logs.

References: CWE-16: Configuration, OWASP A06:2021 - Security Misconfiguration

V-004 - Missing Browser Permissions Policy

STRATEGIC RECOMMENDATION

The application should implement a web application firewall (WAF) to detect and prevent attacks. The application should also implement a intrusion detection system (IDS) to detect and alert on potential security incidents. The application should also conduct regular security audits and penetration testing to identify and address potential vulnerabilities.

TECHNICAL IMPLEMENTATION (CODE FIX)

```
To fix this vulnerability, the application should be properly configured to use a browser permissions policy. Th

Before:
```python
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return 'Hello, World!'
```
After:
```python
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return 'Hello, World!', {'Permissions-Policy': 'geolocation=(), microphone=()'}
```
Server Config:
```apache
Header always set Permissions-Policy 'geolocation=(), microphone=()'
```
```

VALIDATION & VERIFICATION STEPS

Step 1: Test the application's browser permissions policy to ensure that it is properly configured.,Step 2: Use a web application scanner to test the application for browser permissions policy misconfiguration vulnerabilities.,Step 3: Test the application's error handling to ensure that it does not reveal sensitive information.,Step 4: Use a penetration testing framework to test the application for browser permissions policy misconfiguration vulnerabilities.,Step 5: Test the application's authentication and authorization mechanisms to ensure that they are properly implemented.,Step 6: Use a vulnerability scanner to test the application for known vulnerabilities.,Step 7: Test the application's input fields to ensure that they are properly limited and validated.,Step 8: Use a web application firewall (WAF) to detect and prevent attacks.,Step 9: Test the application's logging and monitoring mechanisms to ensure that they are properly implemented.,Step 10: Use a security information and event management (SIEM) system to monitor and analyze the application's security logs.

References: CWE-16: Configuration, OWASP A06:2021 - Security Misconfiguration

V-005 - Insecure TLS Configuration

STRATEGIC RECOMMENDATION

The application should implement a web application firewall (WAF) to detect and prevent attacks. The application should also implement a intrusion detection system (IDS) to detect and alert on potential security incidents. The application should also conduct regular security audits and penetration testing to identify and address potential vulnerabilities.

TECHNICAL IMPLEMENTATION (CODE FIX)

```
To fix this vulnerability, the application should be properly configured to use a secure TLS configuration. The

Before:
```python
import ssl
context = ssl.create_default_context()
```
After:
```python
import ssl
context = ssl.create_default_context()
context.options ⊨ ssl.OP_NO_TLSv1 | ssl.OP_NO_TLSv1_1
```
Server Config:
```apache
SSLProtocol -ALL +TLSv1.2 +TLSv1.3
```
```

VALIDATION & VERIFICATION STEPS

Step 1: Test the application's TLS configuration to ensure that it is properly configured.,Step 2: Use a web application scanner to test the application for TLS configuration vulnerabilities.,Step 3: Test the application's error handling to ensure that it does not reveal sensitive information.,Step 4: Use a penetration testing framework to test the application for TLS configuration vulnerabilities.,Step 5: Test the application's authentication and authorization mechanisms to ensure that they are properly implemented.,Step 6: Use a vulnerability scanner to test the application for known vulnerabilities.,Step 7: Test the application's input fields to ensure that they are properly limited and validated.,Step 8: Use a web application firewall (WAF) to detect and prevent attacks.,Step 9: Test the application's logging and monitoring mechanisms to ensure that they are properly implemented.,Step 10: Use a security information and event management (SIEM) system to monitor and analyze the application's security logs.

References: CWE-295: Improper Certificate Validation, OWASP A03:2021 - Injection

6. STRATEGIC RECOMMENDATIONS

Domain Name

Effort: Medium

Recommendation: The application should implement a domain name system (DNS) to properly configure its domain name. The application should also implement a web application firewall (WAF) to detect and prevent attacks. The application should also conduct regular security audits and penetration testing to identify and address potential vulnerabilities.
Business Benefit: The implementation of a DNS and WAF can help to prevent attacks and protect the application's security. The conduct of regular security audits and penetration testing can help to identify and address potential vulnerabilities.
Tools Required: N/A

7. SECURITY PROGRAM IMPROVEMENTS

Phase	Initiative	Expected Outcome
Short-Term	Conduct a security audit and penetration testing to identify and address potential vulnerabilities.	The identification and addressing of potential vulnerabilities can help to improve the application's security posture.
Medium-Term	Implement a web application firewall (WAF) to detect and prevent attacks.	The implementation of a WAF can help to prevent attacks and protect the application's security.
Long-Term	Conduct regular security audits and penetration testing to identify and address potential vulnerabilities.	The conduct of regular security audits and penetration testing can help to identify and address potential vulnerabilities and improve the application's security posture.

6. OVERALL ASSESSMENT VERDICT

The application's security posture is currently inadequate, and immediate attention is required to address the identified vulnerabilities. The application should implement a web application firewall (WAF) to detect and prevent attacks, and conduct regular security audits and penetration testing to identify and address potential vulnerabilities. The application's risk score is 70, which indicates a high risk of exploitation.

7. TECHNICAL APPENDIX

V-001 - Sqli Boolean

Finding ID: V-001
Endpoint(s): http://zero.webappsecurity.com/
Observed response patterns: See raw scan output

V-002 - Information Disclosure via HTTP Headers

Finding ID: V-002
Endpoint(s): http://zero.webappsecurity.com/
Observed response patterns: See raw scan output

V-003 - Http Security Headers Misconfiguration

Finding ID: V-003
Endpoint(s): http://zero.webappsecurity.com/
Observed response patterns: See raw scan output

V-004 - Missing Browser Permissions Policy

Finding ID: V-004
Endpoint(s): http://zero.webappsecurity.com/
Observed response patterns: See raw scan output

V-005 - Insecure TLS Configuration

Finding ID: V-005
Endpoint(s): http://zero.webappsecurity.com/
Observed response patterns: See raw scan output
END OF REPORT